

IHK AP2 2025 – Vorbereitungs-Handbuch

Entwicklung & Umsetzung von Algorithmen (mit Theorie)

Java-ähnlicher Pseudocode (ohne Datentypen). Jede Sektion ist eine eigene Druckseite: Erklärung · Schritt-für-Schritt · IHK-Stil-Beispiele · Tipps · typische Fehler.

A1) Variablen & Arrays (1D, 2D) – Basics

Fundament

1D: Linear durchlaufen: Index $0 \dots a.length-1$. **2D:** Verschachtelt: äußere = Zeilen/Monate, innere = Spalten/Tage.

```
funktion sum1D(a)
    sum = 0
    for (i = 0; i < a.length; i++) sum = sum + a[i]
    return sum
```

```
funktion sum2D(m) // m: matrix
    sum = 0
    for (r = 0; r < m.length; r++)
        for (c = 0; c < m[r].length; c++)
            sum = sum + m[r][c]
    return sum
```

IHK-Stil Beispiel: Jahresverbrauch $12 \times$ Tage $\rightarrow$ pro Monat summieren (siehe A4).

Fehler: Innere Länge falsch (immer `m[r].length` statt `m.length`), Index außerhalb.

A2) Kontrollstrukturen (if, for, while)

Pflicht

```
funktion countGreater(a, x)
  count = 0
  for (i = 0; i < a.length; i++)
    if (a[i] > x) count++
  return count
```

```
funktion firstIndex(a, x)
  i = 0
  while (i < a.length && a[i] != x) i = i + 1
  if (i == a.length) return -1
  return i
```

Merke: Rückgabe außerhalb der Schleifen, außer bei „sofort gefunden“ (Search).

A3) Zählen, Summieren, Durchschnitt

sehr häufig

Kernpattern: 1× Schleife, Bedingung prüfen, Zähler/Akkumulator anpassen. Durchschnitt = sum / n .

```
funktion durchschnitt(a)
  if (a.length == 0) return 0
  sum = 0
  for (i = 0; i < a.length; i++) sum = sum + a[i]
  return sum / a.length
```

IHK-Stil Beispiel: $a=\{22,27,29,23,26\}$, $>25 \rightarrow 3$; $\emptyset=25,4$.

Fehler: Division durch 0, Rückgabe in der Schleife.

A4) Min/Max (1D & 2D) + Objektrückgabe

Top-Thema

1D

```
funktion findeMin(a)
  min = a[0]
  for (i = 1; i < a.length; i++)
    if (a[i] < min) min = a[i]
  return min
```

```
funktion findeMax(a)
  max = a[0]
  for (i = 1; i < a.length; i++)
    if (a[i] > max) max = a[i]
  return max
```

2D + Objekt

```
funktion ermittleJahresstatistik(verbrauch)
  min = MAX_WERT; max = MIN_WERT
  for (m = 0; m < verbrauch.length; m++)
    summe = 0
    for (t = 0; t < verbrauch[m].length; t++)
      summe = summe + verbrauch[m][t]
    if (summe < min) min = summe
    if (summe > max) max = summe
  return new Jahresstatistik(min, max)
```

IHK-Stil Beispiel (Sommer 2022): Monatsverbräuche → min/max Monats-Summe → Jahresstatistik zurückgeben.

Merke: Für Index zusätzlich minIndex/maxIndex pflegen.

A5) Lineare Suche

häufig

```
funktion linearSearch(a, x)
  for (i = 0; i < a.length; i++)
    if (a[i] == x) return i
  return -1
```

IHK-Stil: $a=\{4700,4711,4800\}$, $x=4711 \rightarrow 1$; sonst -1 .

A6) Binäre Suche (nur sortiert)

seltener

```
funktion binarySearch(a, x)
  left = 0; right = a.length - 1
  while (left <= right)
    mid = (left + right) / 2
    if (a[mid] == x) return mid
    if (a[mid] < x) left = mid + 1 else right = mid - 1
  return -1
```

IHK-Stil: $a=\{2,4,7,9,11\}$, $x=7 \rightarrow 2$. Voraussetzung: sortiert!

A7) BubbleSort

möglich

```
funktion bubbleSort(a)
  for (i = 0; i < a.length - 1; i++)
    for (j = 0; j < a.length - 1 - i; j++)
      if (a[j] > a[j+1]) swap(a[j], a[j+1])
```

IHK-Stil: $a=\{5,2,4\} \rightarrow \{2,4,5\}$. Fehler: Grenze $n-1-i$ falsch.

A8) InsertionSort

möglich

```
funktion insertionSort(a)
  for (i = 1; i < a.length; i++)
    key = a[i]
    j = i - 1
    while (j >= 0 && a[j] > key)
      a[j+1] = a[j]
      j = j - 1
    a[j+1] = key
```

IHK-Stil: $a=\{5,2,4\} \rightarrow \{2,4,5\}$. Fehler: $a[j+1]=key$ vergessen.

A9) Prüfziffern (EAN/ISBN, Luhn)

sehr prüfungsnah

EAN/ISBN-13 (1/3 · Mod-10)

```
funktion pruefzifferEAN13(z12)
  sum = 0
  for (i = 0; i < 12; i++)
    z = z12[i]
    w = (i % 2 == 0) ? 1 : 3
    sum = sum + z * w
  p = (10 - (sum % 10)) % 10
  return p
```

Luhn (Kartenprüfung)

```
funktion luhnGueltig(s)
  sum = 0; parity = s.length % 2
  for (i = 0; i < s.length; i++)
    d = s[i]
    if (i % 2 == parity)
      d = d * 2
      if (d > 9) d = d - 9
    sum = sum + d
  return (sum % 10) == 0
```

IHK-Stil: $p = (10 - (S \bmod 10)) \bmod 10$. Fehler: falsche Parität oder zweites %10 vergessen.

B10) Unit-Test (Definition & Zweck)

Theorie

Definition: Test einer kleinsten Einheit (Funktion/Methode) isoliert.

- Zweck: früh Fehler finden, Verhalten dokumentieren, Regression absichern.
- Muster: AAA – Arrange, Act, Assert.

Formulierung (IHK): „Unittests prüfen einzelne Methoden isoliert. Mit AAA wird erwartetes Verhalten automatisch verifiziert.“

Whitebox – Vorteile

- Fehler exakt lokalisierbar.
- Hohe Abdeckung (Anweisung/Zweig) messbar/steuerbar.

Blackbox – Vorteile

- Prüft Spezifikation (Anforderungen) aus Benutzersicht.
- Technologie-/Implementierungs-unabhängig.

Antwort kurz & präzise, je 2 Vorteile reichen.

B12) Regressionstest

Theorie

Zweck: Nach Änderungen sicherstellen, dass bestehende Funktionen noch korrekt sind.

- Bei Bugfix, Refactoring, Erweiterung, Release.
- Alte Testfälle erneut laufen lassen, ggf. erweitern.

Formulierung (IHK): „Regressionstests verhindern das Wiederauftreten alter Fehler nach Änderungen.“

B13) Abdeckung: Anweisung, Zweig, Pfad

Theorie

- **Anweisung:** Jede Anweisung mind. 1× ausführen.
- **Zweig:** Jeden true/false-Zweig mind. 1×.
- **Pfad:** Alle Pfade (praktisch schwer, Schleifen).

Merke: Pfad $\Rightarrow$ Zweig $\Rightarrow$ Anweisung (Implikation).

B14) Rekursion vs. Iteration

Theorie

- **Pro Rekursion:** oft einfacher/übersichtlicher (Bäume, Teile-und-Herrsche).
- **Contra Rekursion:** Stack-Overflow, Overhead.
- **Pro Iteration:** effizienter bzgl. Speicher/Laufzeit.

Antwortbeispiel: „Vorteil Rekursion: kompakt & natürlich bei Bäumen. Nachteil: Stack-Gefahr; iterativ oft effizienter.“

B15) Generische Klassen

Theorie

Definition: Klassen/Container mit Typparameter – wiederverwendbar & typsicher.

```
klasse Liste<T>  
  add(t: T)  
  get(i): T
```

Beispiel: Liste<Artikel>, Liste<Zahl> – gleicher Code, unterschiedliche Typen.

B16) REST & HTTP-Methoden (CRUD)

Theorie

- **REST:** Ressourcen mit URL, zustandslos, lose Kopplung.
- **CRUD→HTTP:** Create=POST, Read=GET, Update=PUT/PATCH, Delete=DELETE.

GET	/kunden/42	-- lesen
POST	/kunden	-- anlegen
PUT	/kunden/42	-- ersetzen/ändern
PATCH	/kunden/42	-- teilweise ändern
DELETE	/kunden/42	-- löschen

IHK-Frage: „CRUD den HTTP-Methoden zuordnen“.

Spickzettel – Kurzformeln & Antworten

- **Regression:** Nach Änderung prüfen, dass Altes noch funktioniert.
- **Whitebox Vorteile:** Fehler lokalisierbar, hohe Abdeckung.
- **Blackbox Vorteile:** Anforderungssicht, unabhängig vom Code.
- **Abdeckung:** Pfad $\Rightarrow$ Zweig $\Rightarrow$ Anweisung.
- **Rekursion $\pm$:** +einfach bei Bäumen, –Stack/Overhead.
- **Generics:** Typsicherheit + Wiederverwendung (z. B. Liste<T>).
- **CRUD $\rightarrow$ HTTP:** C=POST, R=GET, U=PUT/PATCH, D=DELETE.
- **EAN-Prüfziffer:** $(10 - (\text{Summe} \% 10)) \% 10$.

Druck: Browser $\rightarrow$ Drucken $\rightarrow$ Als PDF speichern. 1 Thema = 1 Seite.